Exact Integer Alias Method: Robust and Error-Free Large-Scale Sampling

Oriane Crouzet, Antoine Genitrini, and Mehdi Naima

Sorbonne Université, CNRS, LIP6, F-75005 Paris, France
Oriane.Crouzet@lip6.fr, Antoine.Genitrini@lip6.fr, Mehdi.Naima@lip6.fr

Abstract. Walker's Alias Method, made linear by Vose, is the gold standard for sampling from a fixed discrete distribution in constant time, but its reliance on floating-point arithmetic makes it unreliable once weights exceed double-precision resolution, a regime already reached by applications such as combinatorial chemistry. Existing exact remedies either rely on arbitrary-precision rational fractions, at a substantial cost in time and memory, or, like the Rust `weighted_rand` crate, on rescaled integer weights, without any published correctness proof. We introduce the Integer-Alias Sampler, a construction and sampling scheme that operates natively and exclusively on arbitrary-precision integer weights. We prove its correctness, give tight bounds on the resulting table size, and analyze the random-bit consumption of the associated generator. Experiments, including a large-scale combinatorial-chemistry case study, show that once weights genuinely require arbitrary precision, our sampler is faster than the other exact Alias variants at comparable memory cost, while the entropy-optimal samplers, which consume fewer random bits, require more memory and are faster only on low-entropy distributions.

Keywords: Alias method · Exact random sampling · Arbitrary-precision arithmetic · Discrete random variate generation.

1 Introduction

Sampling a discrete object from a prescribed, possibly non-uniform, distribution is a recurring primitive in computational science, underlying Monte Carlo simulation, agent-based modeling, and the stochastic components of modern machine learning pipelines, where billions of independent draws must be produced under strict time and memory budgets. The related question of generating or sampling from a large, implicitly described universe of discrete structures has likewise been studied extensively, from the classical treatises of Nijenhuis and Wilf [9] and Kreher and Stinson [4] to Ruskey's survey [10], more recent developments [2], and Skiena's algorithmic toolbox [13]. In practice, the objects to be generated are rarely equally likely: they carry integer or rational *weights*, and an unbiased sampler must reproduce the corresponding distribution exactly, however skewed it is.

The reference solution to this weighted sampling problem is Walker's *Alias Method* [17,18]: it reorganizes a set of n unequal probabilities into n buckets

of equal capacity, each holding at most two outcomes, a primary value and its designated *alias*, so that a single bucket, drawn uniformly at random, together with one biased coin flip, suffices to recover the target distribution. Vose later showed how to build this table in linear time [16], and Smith and Jacobson provided a thorough analysis of the resulting sampler [14]. By front-loading the whole complexity of the distribution into a one-time linear-time preprocessing step, the Alias Method achieves $O(1)$ sampling independently of n, making it the gold standard whenever a fixed distribution must be queried a very large number of times. As for every sampler considered here, a one-time preprocessing is followed by a sampling step repeated as often as needed, each with its own cost.

This efficiency, however, is traditionally obtained at the price of floating-point arithmetic, and modern computational-science workloads increasingly expose the fragility of this choice. Both the construction of the table and the sampling step rely on repeatedly comparing and renormalizing probabilities represented as IEEE 754 doubles, so that rounding drift can silently corrupt the classification of outcomes during preprocessing, and bit-creep can corrupt the coin-flip threshold during sampling. As soon as the total weight of a distribution exceeds 2^{53}, the granularity of double-precision floats becomes coarser than the weight of individual items, which then acquire a *structurally zero* probability of ever being drawn. Such regimes are no longer isolated cases: in combinatorial chemistry, the space of chemically plausible molecules grows so quickly with the number of atoms that it must be sampled rather than enumerated, with reference libraries already ranging from about 10^8 molecules for the smallest structures to several trillion compounds for modern make-on-demand libraries [15], well beyond 2^{64} for molecules of about thirty heavy atoms. Monterrubio-Chanca and von Rudorff [7], whose open-source `nablachem` package [8] we use in our own experiments, address this regime with a pipeline whose final step is precisely a weighted random draw among combinatorial groups, over weights of several hundred bits, well beyond the reach of a double-precision mantissa.

Several directions have been explored to make the Alias Method robust to this kind of scale. The most direct remedy is to abandon floating-point numbers altogether and represent probabilities as arbitrary-precision rational fractions (when it is possible), which restores mathematical exactness but incurs a substantial overhead in both running time and memory, since every comparison and table entry now manipulates a pair of big integers. An alternative, adopted by the Rust ecosystem's `weighted_rand` crate [11], is to restrict the input to non-negative integer weights, instead of probabilities, so that the whole construction can be carried out with exact integer arithmetic; this avoids fractions entirely, but its construction departs from Vose's and its correctness had, to the best of our knowledge, never been established in the literature, and, as we show in Section 3, it also produces alias tables with systematically fewer alias-free cells than Vose's construction, which slows down the sampling step. Orthogonally, entropy-optimal samplers such as FLDR [12] and its refinement ALDR [1] build on the discrete-distribution-generating trees of Knuth and Yao [3] to approach

the information-theoretic minimum number of random bits per sample, at the cost of a preprocessing and sampling cost that grows with the entropy of the distribution, again without native support for arbitrary-precision weights.

In this paper we introduce the *Integer-Alias Sampler*, a construction and sampling scheme that operates natively and exclusively on arbitrary-precision integer weights, without ever resorting to floating-point numbers or rational fractions. We revisit Walker and Vose's construction in this exact integer setting, present the key-ideas for proving the correctness of the Rust integer-weight variant [11], and prove that our own construction, built around a natural integer cell size, yields alias tables with more alias-free cells, and hence a faster sampling step, together with matching tight bounds on the size of the resulting table (Sections 2 and 3). We design an efficient random generation procedure on this table, together with a virtual-object rejection mechanism, and bound its average random-bit consumption, which we then compare experimentally with the near-optimal consumption of the entropy-optimal samplers of [12,1]. We implement all of the above under arbitrary-precision arithmetic, coupled with a background thread that continuously refills a cryptographically secure random-bit stream, overlapping bit generation with sampling, supporting distributions whose number of objects and individual weights can both grow without any fixed-width bound. Finally, we experimentally compare our approach against the Fraction-Alias sampler, the Rust-Alias sampler [11], and arbitrary-precision adaptations of FLDR [12] and ALDR [1], including a large-scale combinatorial-chemistry case study derived from [7,15], showing that once weights genuinely require arbitrary precision, our sampler offers a favorable trade-off between sampling time, memory footprint, and random-bit consumption: it is faster than the other exact Alias variants at comparable memory cost, while FLDR and ALDR consume fewer random bits at a higher memory cost (Section 4).

2 Alias algorithm beyond floating numbers rounding errors

The *Alias Method*, designed by Walker in consecutive papers [17,18], represents a fundamental shift in how we sample from discrete probability distributions. Walker's geometric approach transforms a set of n unequal probabilities into n uniform buckets of equal capacity, each containing at most two outcomes, a primary value and its designated *alias*, by reallocating the surplus from high-probability outcomes to fill the deficits of low-probability ones. Sampling then operates in constant time, regardless of the number of possible outcomes, requiring only a uniformly drawn bucket and a single biased coin flip to pick between its two outcomes.

While Walker's approach is elegant, its efficiency is largely attributed to Vose's 1991 refinement [16], which introduced a linear-time construction of the bucket table (a phase we call *preprocessing* in what follows). Vose achieves this by maintaining two worklists (e.g., stacks) classifying outcomes as small or large relative to the mean, and systematically pairing underfilled with overfilled buckets,

improving on Walker's original super-linear construction. The combination of Walker's $O(1)$ sampling and Vose's $O(n)$ preprocessing makes the Alias Method the reference method whenever a fixed distribution must be sampled a very large number of times, with a decisive advantage over binary-search-based methods as the number of objects grows.

The practical implementation of the Alias Method, however, must contend with finite-precision floating-point arithmetic, which introduces subtle vulnerabilities in both phases. During preprocessing, the iterative normalization and reallocation of probabilities can lead to precision drift, causing the total probability mass to deviate from unity or outcomes to be misclassified between worklists. During sampling, the same imprecision can corrupt the coin-flip threshold: even a microscopic deviation, such as a probability intended to be exactly 1.0 stored as an infinitesimally smaller fraction, can erroneously trigger the alias outcome, while the discrete granularity of the underlying pseudo-random number generator can further distort the intended distribution over the course of massive Monte Carlo trials.

To address rounding errors in both the preprocessing and simulation phases, the most effective approach is to replace standard floating-point arithmetic with arbitrary-precision fractions. In practice, since the floating number have finite precision, the probabilities can be represented as rational numbers, i.e. as fractions of arbitrary size: to that order, the distribution of the objects is based on some non-negative integer weights. This restriction is directly reflected in the probability distribution of the objets, because the probability of any given object is then a rational number. This technique effectively treats the input as a mathematically perfect probability distribution (without any consideration on approximations in the inputs), eliminating the precision drift and bit-creep that typically plague massive Monte Carlo trials. By maintaining exact rational representations, the software ensures that the statistical integrity of the simulation remains uncompromised, regardless of the number of iterations performed. Adapting Vose's preprocessing to exact fractions follows the same two-stack classification scheme as the general integer construction we present in Algorithm 1 (there parametrized by an integer cell size `cs` rather than by fractional probabilities); since no rounding error occurs, the resulting code is in fact simpler than Vose's original implementation. However, relying on exact rational arithmetic introduces significant computational overhead and increased memory usage compared to native floating-point types, which impact overall performance in large-scale simulations.

To improve this aspect, developers for the Rust programming language proposed an alternative approach [11], also in the case if a distribution relying on some integer weights. Rust's adaptation consider a sequence $X = [x_0, x_1, \ldots, x_{n-1}]$ of n distinct objects and a sequence $W = [w_0, \ldots, w_{n-1}]$ of non-negative integers such that object x_i is of weight w_i. We denote the total weight by $w = \sum_i w_i$.

We aim at generating at random an object x_i according to the global distribution W, i.e. the probability for generating x_i is given by w_i/w, for each i. Obviously the later distribution is a probability distribution. Say differently, a

sequence of objects with rational probabilities can be equivalently represented with non-negative integer weights replacing each probability p_i by the integer $w_i = p_i \cdot LCM$ where LCM is the least common multiple of the denominators of the probabilities $(p_i)_i$.

Only the preprocessing phase of the Rust implementation is distinct from the original Alias method, the sampling phase remains identical. We put together our approach and Rust's in Algorithm 1 for a better comparison. Note that the Rust implementation takes advantage of polymorphism, hence the same implementation should apply either for floating point numbers, but also for integer weights distributions. So in the following we simplify a little the pseudo-code for focusing only on necessary parts used for an integer distribution weights for objects.

Algorithm 1 Building of the Alias Table by Rust language and by our approach

Input: X, W, n and w previously defined.
Output: The table of triplets T.

function TABLE_RUST(X, n, W, w)
  $T = \texttt{init}(n)$
  for $x \in X$ **do**
    $W[x] = n \cdot W[x]$
  $L = \{x \in X \mid W[x] \geq w\}$
  $S = \{x \in X \mid W[x] < w\}$
  $k = 0$
  while S is not empty **do**
    $x = \texttt{pop}(L)$
    $x' = \texttt{pop}(S)$
    $T[k] = (W[x'], x', x)$
    $W[x] = W[x] - (w - W[x'])$
    if $W[x] < w$ **then**
      $\texttt{push}(S, x)$
    else
      $\texttt{push}(L, x)$
    $k = k + 1$
  while L is not empty **do**
    $x = \texttt{pop}(L)$
    $T[k] = (w, x, \emptyset)$
    $k = k + 1$
  return T

- $\texttt{init}(n)$: Initialization of a table indexed from 0 to $n - 1$
- L is the stack of the larger objects; S is the stack of the smaller objects
- $\texttt{pop}(\cdot)$: pop operation extracts one object from the stack
- $\texttt{push}(\cdot, \cdot)$: push operation adds one object to the stack

Input: $\bar{X}$, $\bar{W}$, $\texttt{cs}, \bar{q}$ previously defined.
Output: The table of triplets T.

function TABLEBUILDING($\bar{X}$, $\bar{W}$, $\texttt{cs}, \bar{q}$)
  $T = \texttt{init}(\bar{q})$
  $L = \{x \in \bar{X} \mid \bar{W}[x] > \texttt{cs}\}$ ▷ *L is the stack of the larger objects*
  $S = \{x \in \bar{X} \mid \bar{W}[x] \leq \texttt{cs}\}$ ▷ *S is the stack of the smaller objects*
  $k = 0$
  while L is not empty **do**
    $x = \texttt{pop}(L)$
    if S is not empty **then**
      $x' = \texttt{pop}(S)$
      if $\bar{W}[x'] < \texttt{cs}$ **then**
        $T[k] = (\bar{W}[x'], x', x)$
        $\bar{W}[x] = \bar{W}[x] - (\texttt{cs} - \bar{W}[x'])$
      else
        $T[k] = (\texttt{cs}, x', \emptyset)$
    else
      $T[k] = (\texttt{cs}, x, \emptyset)$
      $\bar{W}[x] = \bar{W}[x] - \texttt{cs}$
    if $0 < \bar{W}[x] \leq \texttt{cs}$ **then**
      $push(S, x)$
    else if $\bar{W}[x] > \texttt{cs}$ **then**
      $push(L, x)$
    $k = k + 1$
  for $x' \in S$ **do**
    $T[k] = (\texttt{cs}, x', \emptyset)$ ▷ *objects still in S have a remaining weight equal to $\texttt{cs}$*
    $k = k + 1$
  return T

Vose established the correctness of the original algorithm by proving that S cannot be exhausted before L; hence the objects remaining once L is empty have a weight of exactly 1 and require no alias. Rust's implementation differs: after scaling by n, an object whose weight equals the cell capacity w (that is, an object of exactly average weight w/n) is classified as large (put in L). Intuitively, this change should give more possibilities to complete smaller objects, thus the

correctness should not be affected. But to the best of our knowledge, this approach provided by Rust has not been proved correct in the literature. Furthermore, in the implementation the loop condition (line 8) is different `while S and L are not empty`, and also there is third loop, line 22 to empty the stack S. We can prove it is non necessary in the context of integer weights distribution.

Proposition 1. *Rust algorithm is correct for the building of the Alias Table.*

The proof key-ideas are similar than those we present for our algorithm in Theorem 1.

Both tables produced by Vose and Rust have the same number of buckets, but the number of buckets without aliases may differ. Let us provide an extreme case to compare Vose's and Rust's approaches. Consider a set of n objects where all objects have weight 2, except for the first two objects which have respectively weights 1 and 3. The total weight is $w = 2n$, i.e., the average weight is 2. In Vose's approach, the first $n - 2$ objects (all with weight 2) are directly placed into the result table without aliases. In Rust's approach, due to the LIFO policy (Last-In, First-Out for stacks), and the classification of objects of average weight as large, the processing order is different. The largest object is processed after other large objects and each object with weight 2 is used to complete a small object. This lead to a situation where only a single object ends up in the return table without an alias.

Proposition 2. *The number of cells without aliases in the table produced by Rust is smaller than or equal to the one in the table by Vose. Hence, for the sampling step, the approach by Rust is less efficient than the one of Vose sampling.*

Indeed, the more buckets without aliases, the more efficient the sampling step, since an alias-free bucket returns its object directly, while an aliased one requires an extra biased coin flip.

Rust's integer-weight approach thus remains an interesting way to avoid floating-point arithmetic, but it suffers from two drawbacks: the integer weights may grow very large, and the resulting table has fewer alias-free buckets, which impacts both memory and sampling throughput at scale. Note that deciding whether a table with the maximum number of alias-free entries can be constructed is itself NP-complete; we refer the reader to [14] and references therein for this research direction.

For the rest of the paper, we suppose that the weights are positive integers, otherwise we remove all objects x_i of weight 0; furthermore we suppose that $\texttt{GCD}(W) = 1$, replacing W by its normalization $[w_0/\texttt{GCD}(W), \ldots, w_{n-1}/\texttt{GCD}(W)]$ otherwise, which yields the same probability distribution.

Definition 1. *We define the cell size, denoted by* `cs`*, to be the integer* $\lfloor \frac{w}{n} \rfloor$.

Remark. Since W contains n positive integers, we have $\texttt{cs} \geq 1$; note that $\texttt{cs} = 1$ exactly recovers Vose's original setting.

We compute the Euclidean division of w by $\mathtt{cs}$, defining the quotient q and remainder r such that

$$w = q \cdot \mathtt{cs} + r \qquad \text{with} \quad 0 \leq r < \mathtt{cs}. \tag{1}$$

Let us focus on the simple running example where X contains $n = 5$ objects from x_0 to x_4 with respective weights $W = [2, 9, 1, 6, 1]$. The total weight is $w = 19$ and the cell size is $\mathtt{cs} = 3$; thus $q = 6$ and $r = 1$.

If $r > 0$, we define a *virtual object* x_n of weight $w_n = \mathtt{cs} - r$, and set $\bar{X} = [x_0, \ldots, x_{n-1}, x_n]$, $\bar{W} = [w_0, \ldots, w_{n-1}, w_n]$, $\bar{w} = w + w_n$; if $r = 0$, we simply take $\bar{X} = X$, $\bar{W} = W$. In both cases,

$$\bar{w} = \bar{q} \cdot \mathtt{cs} \qquad \text{with either } \bar{q} = q + 1 \text{ or } \bar{q} = q. \tag{2}$$

In our running example, we append a virtual object x_5 of weight 2, giving $\bar{q} = 7$.

We aim at producing a table T of $\bar{q}$ buckets, each one with the same capacity $\mathtt{cs}$ (which reduces to Vose's original construction when $\mathtt{cs} = 1$), whereas Rust's table has n buckets of capacity w. Each bucket is a triplet (v', x', x), where, for our approach, v' is the weight of x' in the bucket and $\mathtt{cs} - v'$ that of its alias x (with $x = \emptyset$ when $v' = \mathtt{cs}$, meaning x' has no alias); for Rust, v' is the weight of x' and $w - v'$ that of x.

Intuitively, we apply Vose's scheme to $\bar{W}$ and $\mathtt{cs}$, with two stacks L (weights $> \mathtt{cs}$) and S (weights $\leq \mathtt{cs}$): (1) the main loop empties L, completing each object of S by a part of an object of L (an object of weight $\mathtt{cs}$ fills a cell alone); (2) we prove that the objects then left in S all have weight exactly $\mathtt{cs}$, each filling a cell alone.

Taking our running example, our algorithm produces

$$T = [(2, x_5, x_3),\ (1, x_4, x_3),\ (3, x_3, \emptyset),\ (1, x_2, x_1),\ (2, x_0, x_1),\ (3, x_1, \emptyset),\ (3, x_1, \emptyset)].$$

3 Construction of our integer Alias method

The notations X, W, n, w, the cell size $\mathtt{cs}$, the virtual object x_n, and $\bar{X}$, $\bar{W}$, $\bar{q}$ are as introduced in Section 2, where Algorithm 1 builds the table T of $\bar{q}$ triplets (v', x', x), with $v' \in [\![1, \mathtt{cs}]\!]$[1] and x, x' two distinct objects of $\bar{X}$ (x possibly $\emptyset$, when $v' = \mathtt{cs}$, meaning x' has no alias).

Let us present our algorithm approach (see right of Algortihm 1) for the preprocessing step. The algorithm proceeds in two phases. First objects are distributed in two stacks L and S, containing respectively the larger objects (whose individual weights are larger than $\mathtt{cs}$) and the smaller objects of weights smaller than or equal to $\mathtt{cs}$. Then (1) the first step of the algorithm is to empty the stack L. To that order, each object of S is completed with a part of a larger object from L if necessary. In the case where the smaller object is of weight $\mathtt{cs}$ it is put alone, without alias, in its cell. Once done, either S is also empty and the algorithm terminates (correctly) or from there (2) we will prove that S will

[1] The notation $[\![a, b]\!]$ stands for the subset of all consecutive integers from a to b.

contain objects whose remaining weight are exactly $\mathtt{cs}$. Hence each object is put without alias in a new cell of T.

Proposition 3. *The size $\bar{q}$ of the table T satisfies $n \leq \bar{q} \leq n + \left\lceil \dfrac{n-1}{\mathtt{cs}} \right\rceil$.*

Proof. By Equations (1) and (2), we have $\bar{q} = \lceil w/\mathtt{cs} \rceil$: indeed, if $r = 0$ then $\bar{q} = q = w/\mathtt{cs}$, and if $r > 0$ then $\bar{q} = q+1 = \lceil w/\mathtt{cs} \rceil$. By definition of $\mathtt{cs} = \lfloor w/n \rfloor$, the Euclidean division of w by n gives $n \cdot \mathtt{cs} \leq w \leq n \cdot \mathtt{cs} + n - 1$. Dividing by $\mathtt{cs} \geq 1$, we get

$$n \leq \frac{w}{\mathtt{cs}} \leq n + \frac{n-1}{\mathtt{cs}}.$$

Since n is an integer, taking the ceiling yields $n \leq \lceil w/\mathtt{cs} \rceil \leq n + \lceil (n-1)/\mathtt{cs} \rceil$. □

In particular $\bar{q} < (1 + \frac{1}{\mathtt{cs}}) \cdot n + 1$, and $\bar{q} \leq n + 1$ as soon as $\mathtt{cs} \geq n - 1$, which is the typical regime of large weights. Note that these bounds for $\bar{q}$ are tight. In fact, if all the weights are equal (thus we are faced to the uniform distribution) once normalized each weight is 1, then $\mathtt{cs} = 1$ and $q = \bar{q} = n$. For the upper bound, fix two positive integers c and $n \geq 2$, and take a set of n objects, such that $n - 1$ of them have weight $c + 1$ and the last one has weight c (so that $\mathtt{GCD}(W) = 1$). The total weight is $w = (c + 1) \cdot n - 1$, thus $\mathtt{cs} = c$, and we get $\bar{q} = \lceil n + (n-1)/c \rceil = n + \lceil (n-1)/\mathtt{cs} \rceil$. Our running example also reaches the upper bound: $n = 5$, $\mathtt{cs} = 3$ and $\bar{q} = 7 = 5 + \lceil 4/3 \rceil$.

Lemma 1. *After each loop, starting Line 6 of Algorithm 1, the average updated weight of the objects still lying either in L or in S is greater than or equal to $\mathtt{cs}$.*

Proof. We proceed by induction on the number of loop iterations. Initially, if $r = 0$ in Equation (1) then $\bar{w} = w$ over n objects, giving average weight $\bar{w}/n = w/n \geq \mathtt{cs}$; if $r > 0$ then $\bar{w} = (q+1)\mathtt{cs}$ over $n+1$ objects, and $n \nmid w$ (otherwise $r = 0$), so $\mathtt{cs} < w/n$ and $w/(n+1) < \bar{w}/(n+1) = (q+1)\mathtt{cs}/(n+1) < (q+1)w/(n(n+1))$; hence $q + 1 > n$ (refining Proposition 3) and the average weight $\bar{w}/(n+1) \geq \mathtt{cs}$ as well. So the invariant holds when entering the loop at Line 6.

Assume it holds for k objects of average weight $\omega \geq \mathtt{cs}$ at the start of an iteration; the total weight decreases by exactly $\mathtt{cs}$ during the iteration. Either an object of S is fully consumed and removed, leaving $k - 1$ objects of cumulative weight $k\omega - \mathtt{cs}$, hence average weight $\geq (k\mathtt{cs} - \mathtt{cs})/(k-1) = \mathtt{cs}$ whenever $k > 1$ (the algorithm terminates otherwise); or S is empty and an object of L, of weight strictly above $\mathtt{cs}$, is only partially consumed, so k objects remain whose total weight, being both at least $k(\mathtt{cs} + 1)$ and a multiple of $\mathtt{cs}$, is at least $(k+1)\mathtt{cs}$, giving average weight $\geq ((k+1)\mathtt{cs} - \mathtt{cs})/k = \mathtt{cs}$. In both cases the invariant is preserved. □

Lemma 2. *At Line 23 of Algorithm 1, all objects still lying in S have a weight equal to $\mathtt{cs}$.*

Proof. Arriving at Line 23, the loop invariant proved in Lemma 1 is still valid, thus the average weight of the objects still lying in S is greater than or equal to $\mathtt{cs}$. Hence we deduce that these remaining objects have all a weight equal to $\mathtt{cs}$. □

Theorem 1. *Function* TableBuilding *in Algorithm 1 is correct.*

The correctness of the approach lies essentially in proving that the algorithm terminates, T being completed. This is a direct corollary of Lemmas 1 and 2.

In the case where we have a virtual object in the alias table T, the approach puts it as the second component in the first cell of T. This behavior is dictated by our stack implementation.

Let us now introduce our sampling algorithm with virtual object rejection.

Algorithm 2 Random sampling with virtual object rejection

Input: T, `cs`, and `virtual_object`.
Output: A random object from $T \setminus \{\texttt{virtual_object}\}$.

```
1  function RandomGenerationOld(T, cs, virtual_object)
2      q̄ = len(T)
3      result = virtual_object
4      while result == virtual_object do
5          i = Uniform(q̄)
6          b = Bernoulli(T[i][0], cs)
7          if b then
8              result = T[i][1]
9          else
10             result = T[i][2]
11     return result
```

The way we implemented Algorithms Uniform and Bernoulli (called in Lines 5 and 6), is related on Lumbroso's paper [6]. He proved their optimality in terms of random bits, not of running time (Section 4 also evaluates a faster variant). Even if both calls are optimal, the overall algorithm is not optimal in the random bits consumption.

Proposition 4. *The consumption of random bits is in average upper bounded by*

$$\begin{cases} \log_2(\bar{q}) + 3 \leq \log_2\big(n + \lfloor \frac{n-1}{\mathtt{cs}} \rfloor\big) + 3 & \text{if } r = 0, \\ \big(1 + \frac{\mathtt{cs}-r}{w}\big)\,(\log_2(\bar{q}) + 3) \leq \big(1 + \frac{1}{n}\big)\,\big(\log_2\big(n + \lceil \frac{n-1}{\mathtt{cs}} \rceil\big) + 3\big) & \text{if } r > 0. \end{cases}$$

We say that the sampled object is rejected t times when the **While** loop (Line 4 in Algorithm 2) is executed $t + 1$ times.

Lemma 3. *Suppose $r > 0$. The probability that the sampled object is rejected t times is*

$$\mathbb{P}(\textit{exactly } t \textit{ rejections}) = \left(\frac{\mathtt{cs} - r}{\bar{w}}\right)^t \left(\frac{w}{\bar{w}}\right).$$

The average number of rejections is equal to $\mathbb{E}(\textit{number of rejections}) = \dfrac{\mathtt{cs} - r}{w}$.

Proof. The case when the sampled object is rejected has probability $\frac{\mathtt{cs}-r}{\bar{w}}$ that corresponds to the ratio of the weight of the virtual object and the total weight. Thus having exactly t rejections has probability $\left(\frac{\mathtt{cs}-r}{\bar{w}}\right)^t$ and then the next object

is valid (is not the virtual object) with probability $\left(\frac{w}{\bar{w}}\right)$. Thus the first statement is proved. To derive the average number of rejections, we have to calculate

$$\mathbb{E}(\text{number of rejections}) = \sum_{t=0}^{\infty} t \left(\frac{\mathtt{cs} - r}{\bar{w}}\right)^t \left(\frac{w}{\bar{w}}\right).$$

The next step is classical and can be derived from the derivative of the geometric series $\sum_{t=1}^{\infty} t x^{t-1} = \frac{1}{(1-x)^2}$ for $|x| < 1$. Hence,

$$\mathbb{E}(\text{number of rejections}) = \frac{w}{\bar{w}} \cdot \frac{\mathtt{cs} - r}{\bar{w}} \cdot \sum_{t=1}^{\infty} t \left(\frac{\mathtt{cs} - r}{\bar{w}}\right)^{t-1} = \frac{\mathtt{cs} - r}{w}.$$

And the proof is complete. □

The proof of Proposition 4 is a direct consequence of Lemma 3 and the fact that each iteration of the **While** loop performs one call to Uniform($\bar{q}$) and one call to Bernoulli, which together consume on average at most $\log_2(\bar{q}) + 3$ random bits [6].

4 Large-scale sampling experiments

This section is built on our code repository, which is available at GITHUB. Note de Antoine : *a compléter* All the competitive methods are implemented in C, using arbitrary precision arithmetic and a flow of cryptographic random bits. A part of the implementation is devoted to reproduce our experimental results. Experiments were conducted on a MacBook Pro (Apple M4 Pro chip, 14-core CPU) with 48GB of unified memory.

4.1 Overview of the experimental approach

This section evaluates the practical performance of the algorithms introduced in Section 3 with a focus on measurable scalability limits, targeting five quantities: preprocessing time, sampling throughput, memory footprint, random-bit consumption, and savings. All computations use exact integer arithmetic, so results are free of floating-point rounding artifacts, and rely on compact data structures that make explicit the core trade-off between entropic efficiency (in bits) and time efficiency.

A distinctive feature of our implementation is native support for arbitrary-precision arithmetic (to the best of our knowledge, the only one in our comparison to provide exact sampling at arbitrary precision). Sampling draws from a background thread that continuously refills a bitstream, enabling the use of expensive cryptographic random bits with a limited impact on performance, since bit production is decoupled from and overlapped with sampling (reported times include its CPU time).

4.2 Case Study: Combinatorial Chemistry

What is following illustrates how exact sampling becomes essential when the underlying combinatorial space is too large, as in several nowadays applications. The chemical space, i.e. the set of all combinatorially plausible stable molecules, is far too vast to be enumerated. Reference databases report sizes ranging from $9.7 \cdot 10^7$ molecules for the smallest structures (e.g., GDB-17, up to 17 heavy atoms) to several trillion compounds for modern make-on-demand combinatorial libraries [15], and beyond roughly thirty heavy atoms the number of possible structures far exceeds $2^{64} \approx 1.8 \cdot 10^{19}$. Consequently, any statistical analysis based on a subset of this space is structurally biased unless the sample is representative of the true underlying distribution.

To address this context, Monterrubio-Chanca and von Rudorff [7] proposed a method for producing unbiased, representative samples of chemical space without ever enumerating it, applicable to molecules containing on the order of 30 heavy atoms. Obviously, this is a regime in which exhaustive enumeration is impossible. The method proceeds in five steps:

1. Generation of all theoretically possible chemical formulas by integer partitioning;
2. Grouping of these formulas by degree sequence;
3. Distinction between "pure" and "non-pure" sequences, depending on whether several elements of the same valence appear;
4. Estimation of the size of each group, either through exact counting or through a heuristic based on the graph's average path length when exact counting proves too costly;
5. Weighted random sampling among the groups, proportional to their estimated size, followed by the sampling of a representative molecule within the chosen group.

We note that the two steps **1.** and **5.** are classical in the context of combinatorics and combinatorial algorithm design. Especially step **5.** is a classical application of the Alias method. All their results are reproducible through their open-source `nablachem` Python package [8].

Using their library, we derive the distribution of chemical molecules containing 32 heavy atoms, among Carbon, Hydrogen, Oxygen, Nitrogen, and Fluorine. The total number of molecules is $n = 20\,484$, and the sum of their weights is about $1.25 \cdot 2^{168} \approx 4.7 \cdot 10^{50}$. Thus in the following benchmarks for the sampling method, we used these parameters, which seem representative of the actual combinatorial chemistry research. Remark that step **5.** cannot be performed without bias in their Python library based on floating-point arithmetic samplers that are not precise enough in this regime. Note that exactness is understood with respect to the supplied weights: when some group sizes are only estimated heuristically (step **4.**), our sampler reproduces this estimated distribution exactly, but does not correct the estimation itself. More details about this case study case be found in our code repository.

4.3 Algorithms with arbitrary precision integers

Following the design of our Integer-Alias Sampler, the next phase focuses on an experimental evaluation of our algorithm against existing methods (adapted with arbitrary precision). We start from the code repository of Saad et al. [12] in order to add our implementation in their comparison: we thus are able to run all their benchmarks. We have adapted the original memory consumption analysis to the precise size used by the data structures built by the preprocessing step.

While Saad et al. compared many literature approaches to emphasize random bit consumption, we retain the three most relevant ones for our constraints (time, memory, and random bits) at arbitrary precision, all exact (floating-point samplers are left out: on 179 of our 201 distributions, they would never draw some objects, whose bucket share nw_i/w is below 2^{-53}):

- FLDR Sampler: Saad et al. [12] that we adapted for arbitrary-precision integers;
- ALDR Sampler: Draper and Saad [1], an improvement of FLDR in the context of random bit consumption, that we also adapted for arbitrary-precision integers;
- Rust-Alias Sampler: Rust Developers, a variant of Walker version coded in Rust with integer arithmetic [11], where we also have modified types to support arbitrary-precision integers;
- Fraction-Alias Sampler: our new exact version of Walker-Alias using exact rational fractions (with arbitrary precision) instead of floating-point numbers;
- our Integer-Alias Sampler: our new algorithmic approach, in two implementations of the same table: *v1* saves random bits with Lumbroso's optimal UNIFORM and BERNOULLI [6]; *v2* saves time with Lemire's UNIFORM [5] and a Bernoulli test on the same 64-bit word (64 bits per draw).

The experimental protocol starts by fixing the distribution size (the number of objects), that we take to be $n = 20\,484$ and the total weight distribution to be the one computed in the context of combinatorial chemistry (see Section 4.2) $w \approx 4.7 \cdot 10^{50}$. The measurements were collected on a set of random distributions following a Dirichlet distribution. Using a Dirichlet distribution allows varying the entropy while maintaining fixed values for parameters n and w. The distribution itself is not of major importance, once we can choose for the average weight of an object or the entropy of the resulting random distribution (e.g. a gaussian distribution of weights would have given the same kind of experimental simulations). The observed entropy corresponds to Shannon entropy. The latter is a central theoretical measure in the context of random sampling. We refer the reader to the seminal paper of Knuth and Yao [3] for a general overview. Essentially, we note that the closer a distribution is to a uniform distribution, the higher its entropy (bounded by $\log_2(n)$). Conversely, a strongly biased distribution (e.g., one element with a very large weight and many elements with a weight of 1) results in lower entropy. For each distribution, among the 200 random ones, 10 millions of objects have been sampled to measure the average sampling time and the average number of random bits consumed.

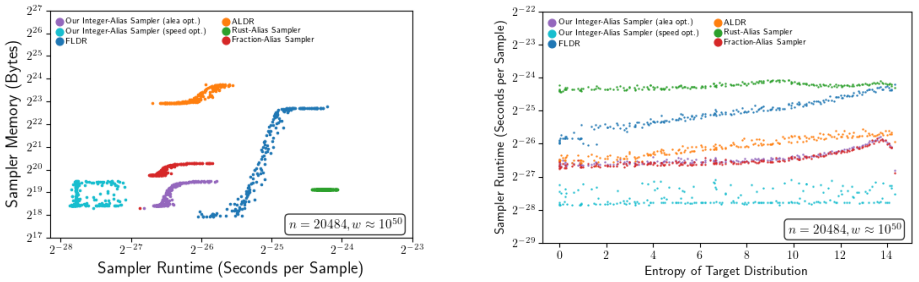


Fig. 1: Sampling runtime under arbitrary precision, against the memory footprint (left) and the entropy of the distributions (right).

Figure 1 (left) plots the runtime of the sampling step against the memory footprint of the internal data structures, one dot per random distribution (averaged over 10 million samples; 2^{-20}s $= 0.95\,\mu$s and 2^{20}B $=$ 1MB). The tight horizontal clustering of Alias-based methods reflects their $O(1)$ sampling step, structurally independent of the distribution, whereas FLDR and ALDR spread much more widely, exposing their sensitivity to its shape. The picture traces an implicit Pareto frontier: FLDR and ALDR are fast but memory-intensive on favorable distributions, while Alias-based approaches remain compact and predictable. Within this family, our Integer-Alias Sampler consistently runs faster than Rust-Alias at comparable memory cost, since normalizing by `cs` rather than by the full cumulative weight w speeds up the sampling step regardless of the distribution; v2 is the fastest of all, 2.3 times faster than v1.

Figure 1 (right) confirms that Alias-based runtimes stay near-constant across the entropy spectrum: the preprocessing step encodes the whole distribution into the table, so each draw only needs a uniform bucket selection and a single Bernoulli test. FLDR and ALDR, built on the DDG-tree framework of Knuth and Yao [3], instead consume random bits one at a time to navigate a binary decision tree whose expected depth equals the distribution's Shannon entropy; they are therefore *faster* at low entropy (skewed distributions yield short, predictable tree paths) and *slower* as entropy approaches $\log_2(n)$ (near-uniform distributions require paths of maximal depth). The crossover, at roughly half the maximum entropy, marks the point where ALDR's entropy-proportional cost exceeds the constant cost of our sampler. The combinatorial-chemistry case study of Section 4.2, being strongly skewed, falls in the low-entropy regime most favorable to FLDR and ALDR; even there, our Integer-Alias Sampler stays within the same order of magnitude in runtime while using strictly less memory.

Regarding memory (Figure 2, top left), Alias-based approaches are more compact than ALDR across all entropies, and than FLDR except at low entropy, where its tree is small (we report the exact footprint of the preprocessed data structure). Within the Alias family, although Rust-Alias's integer weights are n times larger than ours (it normalizes by the total weight w rather than by `cs`), our table needs at most one extra cell (since `cs` $\leq n-1$), so both approaches

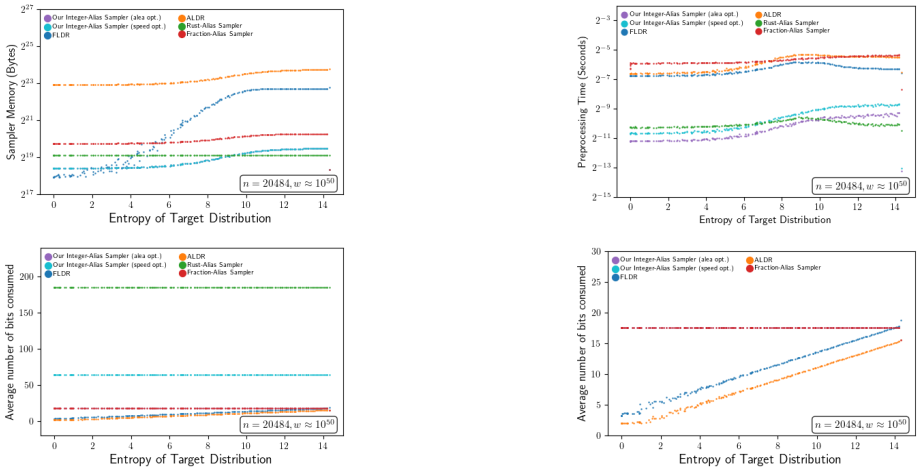


Fig. 2: Memory footprint (top left), preprocessing time (top right), and randomness consumption (bottom) of the compared algorithms under arbitrary precision, against the entropy. The bottom-right plot zooms into the bottom-left one (capped at 30 bits).

use similar memory. This confirms that the runtime gap seen in Figure 1 (left) comes from the arithmetic cost of the Bernoulli step on large integers, not from memory layout. The Fraction-Alias Sampler's table, in contrast, is about twice as large, since it stores a full rational number (numerator and denominator) per entry instead of a single integer.

Figure 2 (top right) shows the preprocessing time (file parsing excluded; the Python construction of FLDR [12] is replaced by an equivalent C port): ours and Rust-Alias's stay below 2.5 ms, twenty times less than FLDR, ALDR (scanning each weight at every tree level) and Fraction-Alias.

Figure 2 (bottom row) reports the average number of random bits consumed per sample, excluding (bottom right) Rust-Alias, whose consumption is an order of magnitude higher, and v2. This is structural: Rust-Alias draws a uniform integer in $[0, w)$, costing $\log_2(w) \approx 153$ bits, instead of a Bernoulli test costing only $\log_2(n) \approx 14$ bits as in our sampler — the direct price of skipping weight normalization, which makes that approach impractical at large precision regardless of its time efficiency.

Among the remaining algorithms, FLDR and ALDR consume fewer random bits than our sampler, with FLDR reaching the information-theoretic optimum of Knuth and Yao [3]; however this advantage narrows as entropy grows, since the dominant cost then becomes the uniform bucket selection ($\log_2(n)$ bits for every algorithm). Near the uniform distribution, this fixed cost dominates and our overhead relative to the optimum becomes negligible: in that regime, our sampler simultaneously achieves $O(1)$ sampling time, a memory footprint among the smallest of the compared methods, and near-optimal bit consumption.

Acknowledgments. This work was supported by the ANR-FWF project PAnDAG ANR-23-CE48-0014 and BpiFrance project Argos.

The authors thank Simon Dreyer for fruitful discussions about these algorithms.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Draper, T., Saad, F.: Efficient rejection sampling in the entropy-optimal range. IEEE Transactions on Information Theory **72**(5), 2801–2822 (2026)
2. Genitrini, A., Pépin, M.: Lexicographic Unranking of Combinations Revisited. Algorithms **14**(3) (2021)
3. Knuth, D., Yao, A.: The complexity of nonuniform random number generation. In: Traub, J.F. (ed.) Algorithms and Complexity: New Directions and Recent Results. pp. 357–428. Academic Press, New York (1976)
4. Kreher, D.L., Stinson, D.R.: Combinatorial Algorithms: generation, enumeration, and search. CRC Press (1999)
5. Lemire, D.: Fast random integer generation in an interval. ACM Transactions on Modeling and Computer Simulation **29**(1), 3:1–3:12 (2019)
6. Lumbroso, J.: Optimal discrete uniform generation from coin flips, and applications. CoRR **abs/1304.1916** (2013)
7. Monterrubio-Chanca, D., von Rudorff, G.: Representative random sampling of chemical space. J. of Chemical Theory and Computation **21**(24), 12810–12819 (2025)
8. Nablachem Lab: nablachem: Python package for representative sampling of chemical space (2025), `https://archive.softwareheritage.org/swh:1:dir:18fd766273c869ddb9012d1e5aca6e65aa377351`
9. Nijenhuis, A., Wilf, H.S.: Combinatorial algorithms. Computer science and applied mathematics, Academic Press, New York, NY (1975)
10. Ruskey, F.: Combinatorial Generation (2003)
11. Rust Programming Language: Weighted_rand (2025), `https://archive.softwareheritage.org/swh:1:dir:41dec5677c27f049668b4fc380183d89db18ca75/`
12. Saad, F., Freer, C., Rinard, M., Mansinghka, V.: Optimal approximate sampling from discrete probability distributions. Proc. ACM Program. Lang. **4**(POPL) (2019)
13. Skiena, S.: The Algorithm Design Manual, Third Edition. Texts in Computer Science, Springer (2020)
14. Smith, J., Jacobson, S.: An analysis of the alias method for discrete random-variate generation. INFORMS J. on Computing **17**(3), 321–327 (2005)
15. Su, J., Song, F., Huang, D., Liao, M.: Constructing and characterizing trillion-scale combinatorial chemical library. J. of Cheminformatics **18**(1) (2026)
16. Vose, M.: A linear algorithm for generating random numbers with a given distribution. IEEE Transactions on Software Engineering **17**(9), 972–975 (1991)
17. Walker, A.: New fast method for generating discrete random numbers with arbitrary frequency distributions. Electronics Letters **10**(8), 127–128 (1974)
18. Walker, A.: An Efficient Method for Generating Discrete Random Variables with General Distributions. ACM Trans. Math. Softw. **3**(3), 253–256 (1977)